

Team 1	Student number	Student name
	e2401834	Karyna Misnik

1. What is Agile software development, and why was it developed? Discuss the main problems in traditional software development that Agile attempts to address.

Agile software development is a software development methodology based on flexibility, short development cycles, and continuous customer feedback. Requirements are considered throughout the development process, and teams work closely together in self-organized teams.

Agile was developed to address problems in traditional software development, such as rigid planning, difficulty handling changing requirements, late feedback, and discovering problems late in the project. Short iterations allow developers to adapt to changes, deliver working software earlier, and receive feedback throughout development.

2. What are the four core values of the Agile Manifesto? Explain each value in your own words and give a practical example of how it could be applied in a software project.

The four core values of the Agile Manifesto are:

- 1. Individuals and interactions over processes and tools** – Good communication and collaboration between people are more important than rigid processes or tools.
Example: A developer discusses a technical problem directly with a teammate instead of waiting for a formal process.
- 2. Working software over comprehensive documentation** – Delivering working software is more important than creating large amounts of documentation.
Example: The team develops and tests a working booking feature instead of spending too much time documenting every detail.
- 3. Customer collaboration over contract negotiation** – Continuous cooperation with the customer is more important than simply following an initial agreement.
Example: The team discusses new customer requirements during development and adjusts the product when necessary.
- 4. Responding to change over following a plan** – Agile accepts that requirements can change, so the team adapts instead of strictly following the original plan.
Example: If the customer requests a new payment method, the team can include it in a future iteration.

3. What are the most important principles of Agile development? Select the five principles you consider most important and explain your choices.

1. Early and continuous delivery of valuable software: deliver useful software early, improve it regularly;

Why important: Customers don't have to wait until the end to get value.

2. Welcome changing requirements: changes are expected at any stage of development;

Why important: The final product can better match what the customer actually needs.

3. Deliver working software frequently: release working software in short periods.

Why important: The early feedback helps to find problems earlier.

4. Business people and developers work together: Customers/business representatives and developers communicate throughout the project.

Why important: Developers understand the real business requirements better.

5. Regularly reflect and improve: The team regularly considers how to work more effectively and adjusts its process.

Why important: The team learns from problems and continuously improves.

4. Compare RUP and Agile. What are their main similarities and differences in terms of planning, documentation, iterations, customer involvement, and requirements?

The similarities that RUP and Agile share are both follow iterative development approaches and both allow feedback. They both welcome changing requirements during development. Their main differences are mostly found in the process structure:

	RUP	Agile
Planning	more structured and detailed planning;	less upfront planning and adjusts plans continuously;
Documentation	more importance on documentation (requirements, use cases, scenarios, and design decisions)	working software and communication over extensive documentation.
Iterations	iterations are often focused on addressing risks and developing the architecture	short iterations, usually 1–4 weeks, to deliver working software and receive feedback
Customer involvement	includes stakeholder feedback, the process is more structured	emphasizes continuous customer involvement and feedback
Requirements	documents and manages requirements carefully and controls changes	expects requirements to change and is designed to respond to those changes more easily

5. Compare the Spiral model and Agile development. How do they deal with risk, change, iteration, and customer feedback?

The Spiral Model and Agile development have similarities such as iterative, they both allow changes during development, involve customers into development to get their feedback, try to identify problems early.

	Spiral Model	Agile
Risk	Risks are identified and analyzed in	Risk is reduced through short iterations,

	every cycle before continuing	testing, and frequent feedback, but risk management is not the central focus
Change	Changes can be made between cycles based on risk analysis and evaluation	Changes are expected and can be applied into future iterations
Iteration	Development happens in repeated cycles of the spiral	Development happens in short iterations/timeboxes
Customer feedback	Customer evaluates the prototype/product during cycles and provides feedback	Customer/stakeholder feedback is frequent and continuous, helping prioritize upcoming work

6. A customer frequently changes the requirements of a software project. Which development model—RUP, Spiral, or Agile—would you recommend? Explain your team's choice.

My recommendation is Agile development. It is designed to be highly adaptive to changing requirements.

Agile uses short iterations, so the team can adjust upcoming work when the customer's needs change.

Close collaboration between the development team and the customer representative helps the team understand which requirements are most important and should be implemented first. Frequent feedback also helps ensure that the software matches the customer's current needs.

RUP and Spiral can also handle changes, but Agile is the most suitable when requirements change frequently because flexibility and continuous customer feedback are central to Agile development.

7. Compare Scrum, Extreme Programming (XP), and Feature-Driven Development (FDD). What are the main characteristics of each model, and when would you choose one over the others?

Scrum, Extreme Programming (XP), and Feature-Driven Development (FDD) are all Agile approaches, but they emphasize different aspects of development.

Scrum main focus is on how development work organized and managed through short Sprints. It uses a Product Backlog, Sprint Backlog, Daily Scrum, Sprint Review, and Retrospective. If a team needs a clear understanding of priorities and collaboration, I would choose Scrum.

Extreme Programming (XP) focuses strongly on software quality and engineering practices. Important practices include pair programming, test-driven development, automated testing, refactoring, and continuous integration. I would choose XP when frequent changes and high software quality are important.

FDD focuses on developing small, client-valued features. It uses domain modeling, feature lists, feature teams, and regular builds. I would choose FDD when the system can be divided into clearly defined features that can be designed and developed separately.

8. What are the main advantages and disadvantages of Scrum? Discuss situations in which Scrum may work well and situations in which it may not be suitable.

Advantages	Disadvantages
Short Sprints - regular working results	Requires active participation from the team
Adaptive to changing requirements	Frequent changes can make long-term planning difficult
Regular customer/stakeholder feedback	Daily meetings and Scrum events require time
Self-organizing teams can make decisions quickly	Scrum may feel restrictive if the work cannot be divided into Sprints
Problems can be discovered early	Requires good communication and collaboration

Scrum works well when:

- requirements may change frequently
- the product can be divided into small pieces of work
- the customer/stakeholder can provide regular feedback
- the team is cross-functional and self-organizing

Scrum is not suitable when:

- requirements are fixed and unlikely to change
- the work cannot easily be divided into Sprint-sized tasks
- customer/stakeholder feedback is not available regularly
- the team cannot work collaboratively or self-organize
- the project requires very detailed long-term planning from the beginning

9. Explain the three main Scrum roles: Product Owner, Scrum Master, and Developers.

What are their responsibilities, and how should they work together?

The three roles work together closely. The Product Owner communicates what should be built and what is most valuable, the Developers decide how to build it, and the Scrum Master helps the whole team work effectively using Scrum.

Product Owner	Maximizes product value and manages the Product Backlog
Scrum Master	Helps everyone understand and follow Scrum; removes obstacles
Developers	Build the product and create the working increment

10. What are Scrum artifacts and events? Explain how the Product Backlog, Sprint Backlog, Increment, Sprint, Daily Scrum, Sprint Review, and Sprint Retrospective contribute to a project.

Scrum artifacts - work items that show what is being worked on and what has been produced.

Scrum events → activities that help the team plan, coordinate, inspect, and improve.

Artifact	What it means	Contribution
Product Backlog	Ordered list of everything that may be needed in the product	Shows what needs to be done and helps prioritize work
Sprint Backlog	Selected Product Backlog items for the current Sprint + plan for delivering them	Shows what the team is working on now
Increment	The completed, working product result of a Sprint	Provides a usable version of the product and allows feedback

Event	What happens	Contribution
Sprint	A fixed period in which Developers work toward a Sprint Goal	Provides a regular development cycle
Daily Scrum	Short daily meeting for Developers to inspect progress and plan the next work	Keeps the team coordinated and exposes problems
Sprint Review	Team and stakeholders inspect the Increment and discuss what to do next	Provides feedback and can change priorities
Sprint Retrospective	Team discusses what went well, what went wrong, and how to improve	Helps the team continuously improve

11. Imagine that your team is developing a mobile application using Scrum. Design one Sprint for the project, including the Sprint Goal, some Product Backlog items, Sprint activities, and expected Increment.

Imagine a team is developing a mobile escape-room booking application.

Sprint Goal:

The goal of the Sprint is to allow users to view available escape rooms and make a basic booking.

Product Backlog items selected for the Sprint:

- View available escape rooms.
- Select a booking date and time.
- Submit a booking.
- Display a booking confirmation.

Sprint activities:

The team plans the Sprint, designs the screens, implements the features, tests the application, and

holds Daily Scrums to coordinate the work. At the end of the Sprint, the team demonstrates the application during the Sprint Review and discusses improvements during the Sprint Retrospective.

Expected Increment:

At the end of the Sprint, the team should have a working mobile application in which a user can view available escape rooms, select a date and time, make a booking, and receive a confirmation.

12. What are the main roles and practices in Extreme Programming (XP)? Discuss how practices such as pair programming, test-driven development, continuous integration, and refactoring improve software quality.

XP typically involves roles such as:

Customer - explains requirements, priorities, and provides feedback.

Developers/programmers - design, code, test, and improve the software.

Coach - helps the team follow XP practices and improve its process.

Tester - helps verify that the software works correctly.

Tracker - monitors progress and helps identify problems.

Quality is checked throughout development, rather than waiting until the end.

13. What are the main advantages and disadvantages of XP? Compare XP with Scrum and discuss which type of project would benefit most from XP.

Advantages of XP	Disadvantages of XP
High software quality	Requires strong team discipline
Finds bugs early through testing	Pair programming can feel expensive/time-consuming
Handles changing requirements well	Requires frequent customer involvement
Frequent integration reduces integration problems	Some practices can be difficult for inexperienced teams
Clean and maintainable code through refactoring	Less suitable when requirements are very stable
Strong communication and knowledge sharing	Can be difficult with large or distributed teams

XP is particularly useful when:

- requirements change frequently
- software quality is very important
- the project needs frequent testing
- developers can work closely together
- the customer can provide frequent feedback
- the codebase is complex and needs continuous improvement

For example, a new mobile application with frequently changing requirements could benefit from XP because TDD, pair programming, CI, and refactoring help the team adapt while maintaining quality.

14. What is Feature-Driven Development (FDD)? Explain its main features and describe how FDD organizes software development around features.

Feature-Driven Development (FDD) is an Agile software development approach that organizes development around small features defined by a customer.

The main features of FDD include domain object modeling, creating a feature list, planning by feature, designing and building by feature, feature teams, regular builds, and software development best practices.

FDD first creates an overall understanding of the system and identifies its main objects and relationships. The system is then divided into a list of small features. These features are prioritized and assigned to developers or feature teams. Each feature is then designed, implemented, tested, and integrated into the system. Regular builds ensure that the software remains working throughout development.

15. Case study: Your team must develop a large online shopping system. Requirements are expected to change frequently, the customer wants regular demonstrations, and the system must be delivered incrementally. **Which development model or combination of models would you choose—Scrum, XP, FDD, RUP, Spiral, or another approach?** Justify your choice and explain how your team would organize the development process.

I would choose Agile development, using Scrum as the main framework and XP practices for technical development.

Since the requirements will be changed frequently Scrum is more suitable in this situation. It is also a good choice if the customers want regular demonstrations of working software. The development would be divided into short Sprints, usually 2–4 weeks. At the beginning of each Sprint, the team would select and prioritize items from the Product Backlog.

During the Sprint, Developers would design, implement, and test the selected features. Daily Scrums would help the team coordinate their work and identify problems early. At the end of the Sprint, the team would demonstrate the working Increment to the customer during the Sprint Review. Customer feedback could then be used to update and reprioritize the Product Backlog for the next Sprint. The team would also hold a Retrospective to improve its development process.

Because this is a large shopping system, the team could organize the Product Backlog around features such as user registration, product search, shopping cart, payment, and order tracking. FDD principles could also be used to organize development around these features.

XP practices such as TDD, continuous integration, pair programming, and refactoring could be used to maintain software quality.